

<예제>

파이토치 기반 ANN, 인공 신경망으로 복잡한 곡선을 학습시키는 예제

[큰 흐름은 다음과 같다.]

1. 라이브러리 불러오기
2. 입력 데이터 X, 정답 데이터 Y
→ 입력은 뭐를 입력? 정답은?
3. ANN 모델 구조 정의
4. 손실 함수와 최적화 방법 정하기
5. 반복 학습하며 모델이 정답 곡선에 가까워 지도록 가중치 수정

1. import torch

→ PyTorch 라이브러리 전체를 불러오는 코드
PyTorch는 인공 신경망을 만들고 학습시키는데 많이 쓰이는 딥러닝 라이브러리.
여기서는 데이터를 Tensor로 바꾸고, 모델을 만들고 역전파를 계산하는데 사용된다.

2. import torch.nn as nn

torch.nn 을 불러온다.

torch.nn 은 PyTorch 안에서 인공 신경망을 만들때 필요한 도구들이 있는 모듈.

as nn 은 이것을 nn으로 부른다는 의미. 코드를 짧고 보기 쉽게 만드는 용도

3. import torch.optim as optim

torch.optim 을 불러오는 것.

torch.optim은 최적화 알고리즘을 모아둔 모듈.

: 최적화 알고리즘이란 모델이 특린 정도를 줄이기 위해 가중치를 어떻게 수정할지 결정하는 방법.

여기 예제에서는 adam 이라는 최적화 방법을 사용한다.

4. import matplotlib.pyplot as plt

matplotlib.pyplot 는 그래프를 그리는 라이브러리이다.

5. import numpy as np

numpy는 숫자계산용 라이브러리

→ 배열 만들기, sin, cos, 랜덤 노이즈 만들기 등에 쓰임.

여기에서는 입력 X_np, 정답 Y_np를 만들때 사용됨.

6. from IPython.display import clear_output

Jupyter Notebook 이나 Colab 에서 출력화면을 지우는 기능을 가져오는 코드.

7. import time

시간 관련 기능을 쓰기 위한 라이브러리.

8. np.random.seed(42)

torch.manual_seed(42)

랜덤 시드 고정하는 것. 처음 가중치 값, 노이즈 데이터.

인공 학습 과정의 목적 위성. 등이 매번 달라지면.

실행 때마다 달라짐. ✓ → 랜덤은 달라져야 하는 것 아닌가?

seed(42)로 고정하면 매번 같은 랜덤 값이

나오도록 설정한 것.

시드가 무엇인지?

시드 왜 42?

(?) 시드?

12. class simplePredictiveAI (nn.Module):

→ class는 파이썬에서 새로운 객체 설계를 만드는 문법이다.

즉, simplePredictiveAI라는 이름의 AI 모델 설계를 만드는 뜻.

→ nn.Module

: PyTorch의 모든 신경망 모델은

nn.Module을 상속받아 만든다.

즉, 작동하기 위한 기본기능을 물려받는다.

쉽게 말하면, 이모델은 PyTorch 신경망 모델

이라고 선언하는 것.

13. def __init__(self):

Super(simplePredictiveAI, self).__init__()

윅! __init__의 초기화 함수.

모델이 처음 만들어질때 한번실행,

여기서는 신경망 층을 정의...

이제 무슨의미인지?

두번째를, nn.Module의 초기화 기능은 실행하는 코드.

→ PyTorch 모델로서 필요한 기본세팅을 먼저해라.

↓

여렇게 해야, 모델 내부의 층과

파라미터를 제대로 인식할수 있다.

14. self.netWork = nn.Sequential(

self.netWork라는 변수에 신경망구조를 저장

: nn.Sequential()은

여러층을 순서대로 쌓는것.

즉 입력이들어오면

Linear → ReLU → Linear → ReLU → ...

15. nn.Linear(1, 64),

nn의 Linear 식은

nn.ReLU(),

이름하는것인가?

nn.Linear(64, 64),

(의미가?)

nn.ReLU(),

nn.Linear(64, 64),

nn.ReLU(),

nn.Linear(64, 64),

nn.ReLU(),

nn.Linear(64, 1),

)

→ 첫번째 입력노드는 1개 → 출력노드 64개

입력변수는 x한개, 1차원.

→ 출력 = 입력 x 가중치 + bias

수식으로 $z = Wx + b$,,

→ nn.ReLU(),

ReLU 활성화 함수이다.

ReLU는 다음과 같다.

input $x \rightarrow \begin{cases} x \text{가 0보다 작으면 } 0 \\ x \text{가 0보다 크면 } x \text{로} \end{cases} \text{ 출력}$

$ReLU(x) = \max(0, x)$ 이다.

23. frames = []

: GIF 애니메이션에 들어갈 이미지 프레임 저장할 빈 리스트

24. for epoch in range(epochs):

: range(epochs) 는 0 ~ 999 까지 반복

25. model.train()

: 모델을 학습모드로 바꾸는 코드

PyTorch 모델에는 학습모드, 평가모드 두가지가 있음.

Dropout 층, Batch Norm 층이 없는 모델이라 큰차이는 없음.

Dropout 층, Batch Norm 층이 무엇인지?

26. y_pred = model(x)

: 모델에게 X 입력을 넣고 예측값을 계산하는 코드

forward() 함수가 실행된다.

처음에는 y_pred가 실제 y와 다름.

순전파 및 오차 계산 (26, 27) 임

27. loss = criterion(y_pred, y)

: y_pred 와 y를 비교해서 Loss 계산하는 코드

Criterion은 앞에서 만든 nn.MSELoss() 이다.

이 코드의 학습 목표는 loss를 줄이는 것이다.

28. # 역전파 및 가중치 업데이트

① optimizer.zero_grad()

② loss.backward()

③ optimizer.step()

①은 이전에서 계산된 gradient를 초기화하는 코드

: PyTorch에서는 loss.backward()를 실행하면 gradient가 계산되고 누적됨.

따라서 매 반복마다 새로 계산하려면 기존 gradient를 지워야 함.

② loss.backward()

: 역전파를 수행하는 코드.

가중치를 어느방향으로 얼마나 바꿔야 하는지 계산하는 과정.

→ 모델내부 모든가중치에 대해 gradient 계산됨.

③ optimizer.step()

실제로 가중치 업데이트하는 코드.

29. loss_history.append(loss.item())

: Loss 값을 loss_history에 저장

loss.item은 Tensor 형태를 일반 숫자로 바꿔내는 함수.

30. if (epoch+1) % 10 == 0 or epoch == 0:

: 첫번째 epoch, 10 epoch 마다 그래프 그려라.

↓

아주에는 그래프 출력에 관련된 코드.

여기서 η 는 업데이트의 정도를 나타내는
하이퍼 파라미터, learning rate 라고 부른다.

결국, 순방향 전파 $\rightarrow$ 손실함수 계산 $\rightarrow$ 역방향 전파
 $\rightarrow$ 학습 (update) 순서대로 진행된다.

* 공부하며 궁금했던 점

2. 시드가 무엇인가?

시드 42를 지정 $\rightarrow$ 이후 랜덤의 순서가 고정됨.

여기서는 데이터에 랜덤 노이즈를 넣을 때
항상 일정한 랜덤 노이즈가 생성이 된다는 것.

실행을 여러번 하더라도, 랜덤 노이즈는 항상 같다.

또한 ANN 모델의 초기 가중치는 랜덤으로 정해지는데
이것도 고정시켜 학습 경로를 일정하게 만드는 것이다.

(즉, 시드 고정 $\rightarrow$ 랜덤 노이즈, 랜덤 초기 가중치 일정.)

2. 실제 데이터를 넣을 때는 어떻게 넣는가?

공식: 없는 데이터는 실험 측정, 기록을 통해
 x (입력) 과 y (출력, 정답) 의 짝을
만들어야 한다.

입력 데이터 x 가 4개라면

```
X_np = np.array([
    [200, 60, 20, -0.02]
    [205, 60, 25, 0.00]
    :
    :
    :
])
```

과같이 X 를 넣고, 이에 대응하는 Y 를

```
Y_np = np.array([
    [1]
    [0]
    [1]
    :
])
```

로 만들어서 짝지어준다.

또한 이에 맞추어

모델 첫 층은

(1,64) 에서 (4,64) 로 바뀌어야 한다.

또한 예제처럼 연속적인 y 라면

MSE Loss 를 사용하며, regression 으로
분류되지만, 이진법 분류 예측 문제일 때에는
(classification)

BCE Loss 를 보통 사용한다.

2-1. BCE Loss ?

Binary cross Entropy loss 의 줄임,

$\rightarrow$ 이진 교차 엔트로피 손실함수

0, 1 이 정답으로 제시된 상황에서 (예시)

모델 예측값이 0.9로 나와면

1 일확률이 90% 0 일확률이 10% 이다.

정답이 1인 상황에서 모델이 0.99에

가깝게 예측하면 Loss가 매우 작고,

0.1에 가깝게 예측하면 Loss를

크게 주는 방식이다.

$$BCE = -[y \log(p) + (1-y) \log(1-p)]$$

p 는 모델 예측 값 (0 ~ 1 사이)

y 는 정답 (0, 1)

정답이 1 이라면

$$BCE = -\log(p) \quad \text{이때 } p \text{가 1에}$$

가까우면 Loss가 작아지는 형식...

정답이 0 이면 반대 일것.

(주의) BCE Loss 사용하면 마지막에는

$\star$ Sigmoid, activation function을

붙임 $\rightarrow$ 0, 1 사이의 값이 나와야 해서.

7. optimizer가 무엇인지? Adam은?

optimizer는 Loss를 줄이기 위해 모델의 가중치와 편향을 어떻게 수정할지 결정하는 알고리즘.

Adam은 그 optimizer 중 하나이며, 비교적 안정적이고, 빠르게 학습시키는 방법.

backwark, 역전파에서 얻은 $-\frac{\partial L}{\partial W}$ 등을 실제로 적용해서 수정하는 알고리즘.

예제에서는 `Optim.Adam(model.parameters(), lr=0.01)`

로 정의

$W_{new} \leftarrow W_{old} - \eta \frac{\partial L}{\partial W}$ 는 SGD 방법 //

Adam은 이에 더 나아가서

- 같은 방향이면 더 안정적으로 수정,
큰 gradient는 조금에서 너무 큰 영향을 미치지 않게 조정,
이라는 특징이 있다.

8. Drop out 층, Batch Norm 층이 뭐지?

• Drop out 층

: 학습중 신경망의 일부 뉴런을 거버리는 (무작위) 방법이다.

→ 과적합을 줄이기 위해!

학습데이터에만 너무 치중, 새로운 데이터에는 약해지는 경우, but drop out이 너무크면 과하다.

• Batch Norm 층

: 각 층으로 들어가는 값들의 분포를 일정하게 맞춰 학습을 안정화 시키는 층.

→ 각 층의 출력값이 크게 바뀌는 경우.

너무큰값은 줄이고,

작은값은 키우고,

적절히 평균과 분산을 안정화한다

단, 데이터가 너무 적으면 불안정,